

Kerberosity Killed the Domain: An Offensive Kerberos Overview

 hausec.com/2020/03/10/kerberosity-killed-the-domain-an-offensive-kerberos-overview

By Hausec

March 10, 2020



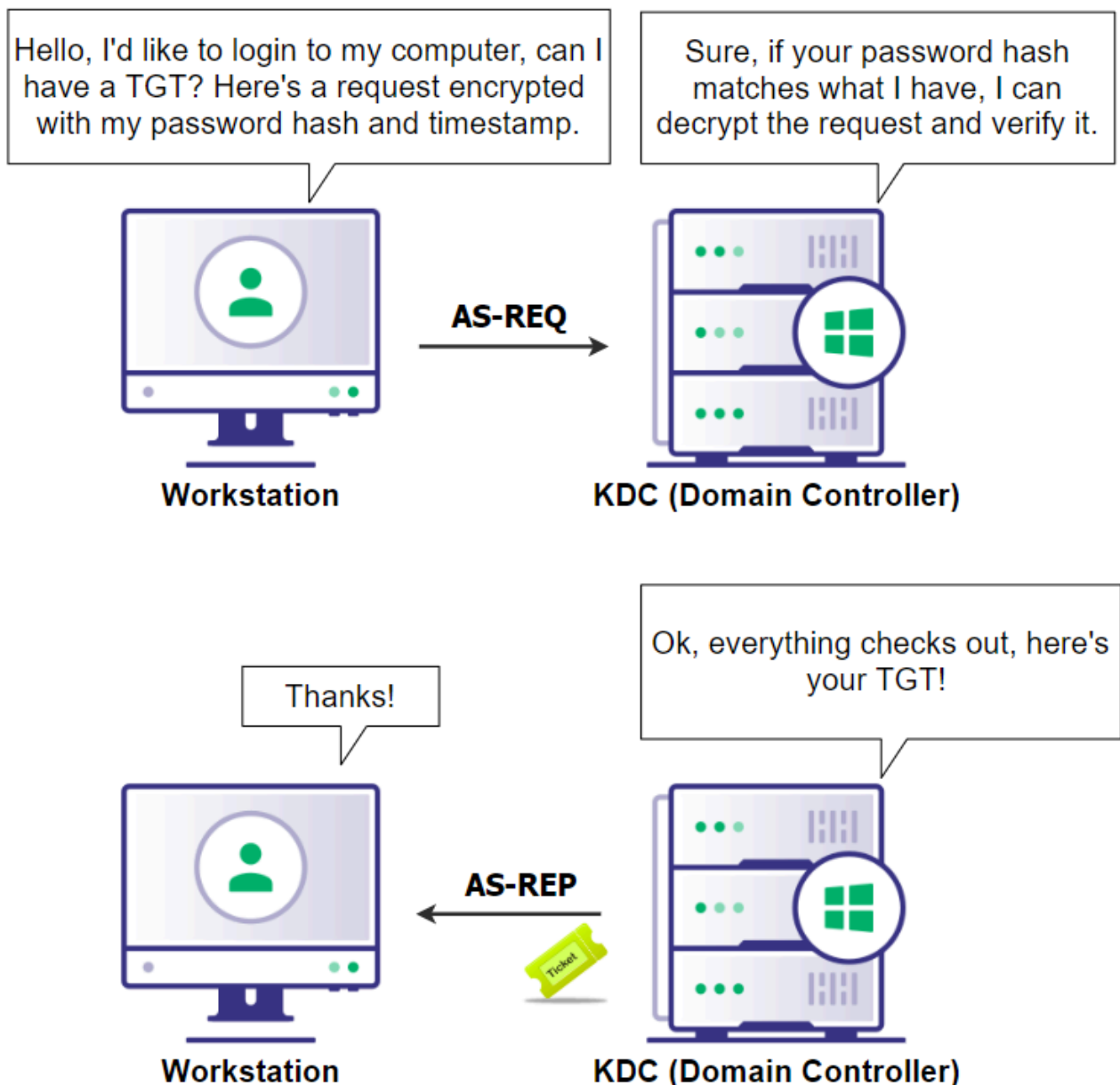
Kerberos is the preferred way of authentication in a Windows domain, with NTLM being the alternative. Kerberos authentication is a very complex topic that can easily confuse people, but is sometimes heavily leveraged in red team or penetration testing engagements, as well as in actual attacks carried out by adversaries. Understanding how Kerberos works legitimately is essential to understanding the potential attack primitives against Kerberos and how attackers can leverage it to compromise a domain. This article is intended to give an overview of how Kerberos works and some of the more common attacks associated with it.

Overview

Kerberos revolves around objects called 'tickets' for authentication. There's two types of tickets:

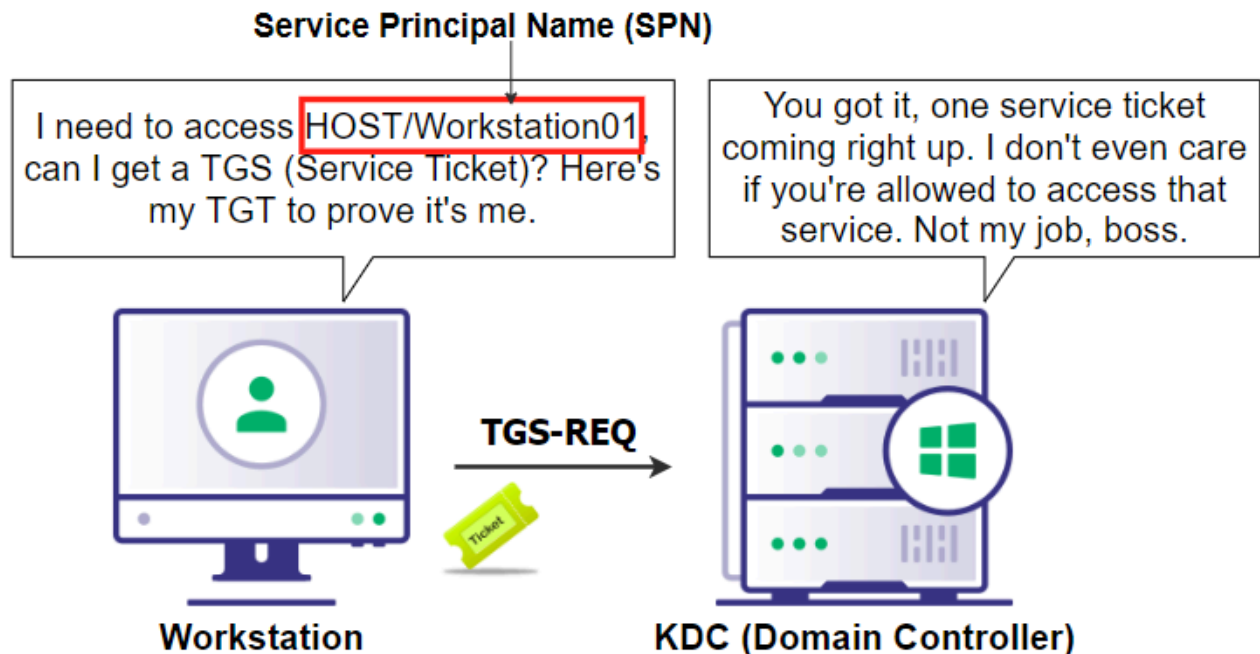
- Ticket-Granting-Ticket (TGT)
- Ticket-Granting-Service (TGS, also called a 'service ticket')

When a user logs into Windows on a domain-joined computer, the password they input is hashed and used as a key to encrypt a timestamp which is sent to the Key-Distribution Center (KDC), which is located on the domain controller. This encrypted timestamp is then sent as an AS-REQ (Authentication Server Request). The KDC then verifies the user's credentials by decrypting the request with the user's password hash in AD and verifying the timestamp is within acceptable limits. The KDC then responds with an AS-REP (Authentication Server Reply).



The AS-REP contains the TGT encrypted with the KRBTGT's key (password hash) as well as some other data encrypted with the user's key. The KRBTGT account is an account that is created when promoting a DC for the first time and is used by Kerberos for authentication. Compromising the KRBTGT account password has very serious implications to it, which will be covered later.

Now that the user is authenticated in the domain, they still need access to services on the computer they're logging into. This is accomplished by requesting a service ticket (TGS) for a *service principal* via TGS-REQ. A service principal is represented through its service principal *name* (SPN). There's many SPNs, a majority of which can be found [here](#). For accessing the actual machine, the SPN 'HOST' is requested. HOST is the principal that contains all the [built-in services](#) for Windows.



A TGS contains a *Privileged Attribute Certificate* (PAC). The PAC is what contains information about the user and their memberships, as shown in figure 3.

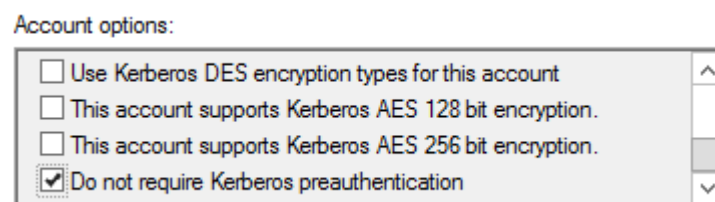
Once the TGS is received, via TGS-REP, the target service decrypts the ticket with it's password hash (in this case, it's the computer account's password hash) and looks in the TGS's PAC to see if the appropriate group SIDs are present, which determines access. The key distinction with service tickets, is that the KDC does the *authentication* (TGT) where the service does the *authorization* (PAC in the TGS). Once confirmed, the user is allowed to access the HOST service principal and the user is then logged into their computer.

This entire logon process can be viewed in Wireshark when capturing a login process for another user.

Time	Source	Destination	Protocol	Length	Info
11.15...	192.168.10.30	192.168.10.10	KRB5	265	AS-REQ
11.15...	192.168.10.10	192.168.10.30	KRB5	222	KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED
11.16...	192.168.10.30	192.168.10.10	KRB5	345	AS-REQ
11.16...	192.168.10.10	192.168.10.30	KRB5	71	AS-REP
11.16...	192.168.10.30	192.168.10.10	KRB5	1447	TGS-REQ
11.16...	192.168.10.10	192.168.10.30	KRB5	1471	TGS-REP
11.22...	192.168.10.30	192.168.10.10	KRB5	1618	TGS-REQ
11.23...	192.168.10.10	192.168.10.30	KRB5	129	TGS-REP

In figure 5, the first AS-REQ is responded with 'KRB Error: KRB5KDC_ERR_PREAUTH_REQUIRED'. Prior to Kerberos version 5, Kerberos would allow authentication [without a password](#). In Version 5, it does require a password, which is called Pre-Authentication. Presumably for backwards compatibility reasons, Kerberos tries to authenticate without a password first, before using Pre-Authentication, which is why there's always an error after the initial AS-REQ during a logon. This leads in to the first attack that will be covered, *AS-REP Roasting*.

AS-REP Roasting



There is a setting in the Account options of a user within AD to not require Kerberos Pre-Authentication.

Since a timestamp encrypted with the user's password hash is used as an encryption key for an AS-REQ, if the KDC successfully reads the timestamp using the user's password hash, as long as the timestamp falls within a few minutes of the KDC's time, it issues the TGT via AS-REP. When Pre-authentication is not required, an attacker can send a fake AS-REQ which the KDC will immediately grant a TGT because there's no password needed for verification. Since part of the AS-REP (apart from the TGT) contains data (a session

key, TGT expiration time, and a nonce) that is encrypted with the user's key (password hash), the password hash is able to be pulled from that request and cracked offline. More can be read [here](#).

In Rubeus, this can be accomplished with the asreproast function.

Kerberoasting

When a TGS is issued, a timestamp + password hash for the service account is used to encrypt the TGS since the password is the shared secret between the service account and the KDC/DC. This is most commonly a service (such as HOST or CIFS) that is controlled by the computer, so the computer account password hash is used. In some cases, user accounts are created to be "service accounts" and registered as a service principal name. Since the KDC does not perform authorization for services, as that is the service's job, any user can request a TGS for any service. This means that if a user "service account" is registered as an SPN, any user can request a TGS for that user which will be encrypted with the user account password hash. That hash can be extracted from the ticket and cracked offline.

With Rubeus, this can be accomplished using the kerberoast function.

Golden Ticket

As briefly mentioned earlier, when a TGT is issued, it is encrypted with the KRBTGT's account password hash. The KRBTGT's password, by default, is never set manually and thus is as complex as a machine accounts password. A golden ticket attack is when the KRBTGT password is compromised and an attacker forges a TGT. The RC4 hash of the KRBTGT password can be used with mimikatz to forge a ticket for any user without needing their password.

```
mimikatz # kerberos::golden /domain:lab.local /sid:S-1-5-21-600110288-2354598277-2662498018 /rc4:19e92d0c3804147837ba1faac7a7c06d /user:ADMBob /id:500 /ptt
```

KRBTGT RC4 Password **Targeted User for the TGT and their RID (500 = Admin)** **SID of domain** **"Pass the ticket" -- Importing it**

Silver Ticket

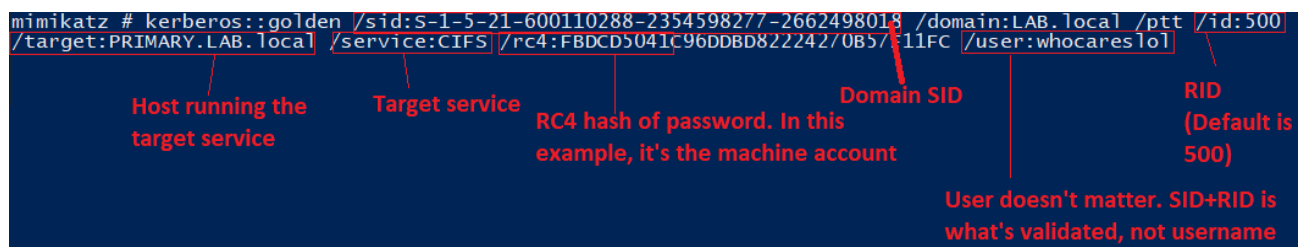
Where a golden ticket is a forged TGT, a silver ticket is a forged TGS. The major opsec consideration with golden tickets is that there is a transaction that occurs within the KDC — a TGT is issued, which allows defenders to alert on these transactions and potentially catch golden ticket attacks. Silver tickets are much more stealthy because they never touch the KDC. Since a service ticket is being forged, knowledge of the target service's password hash is needed, which in most cases will be the machine account password

hash. In the case of service accounts with an SPN set, a silver ticket can be generated for that SPN.

For example, if a service account is created under the username 'MSSQLServiceAcct' and registered for the MSSQLSVC principal, the SPN would be MSSQLSVC/MSSQLServiceAcct. If an attacker obtained that accounts password hash (via Kerberoasting or other means), it could then forge a TGS for that SPN and access the service that utilizes it (MSSQLSVC).

In the case of certain services, such as CIFS, where an SPN for a user account (e.g. CIFS/Alice) is made, a silver ticket for CIFS using the user's password will *not* work because the user does not control access to that service, the machine account does.

In the example shown below, an attacker gained knowledge of the domain controller's computer account hash and generated a silver ticket for CIFS to access it's file system.

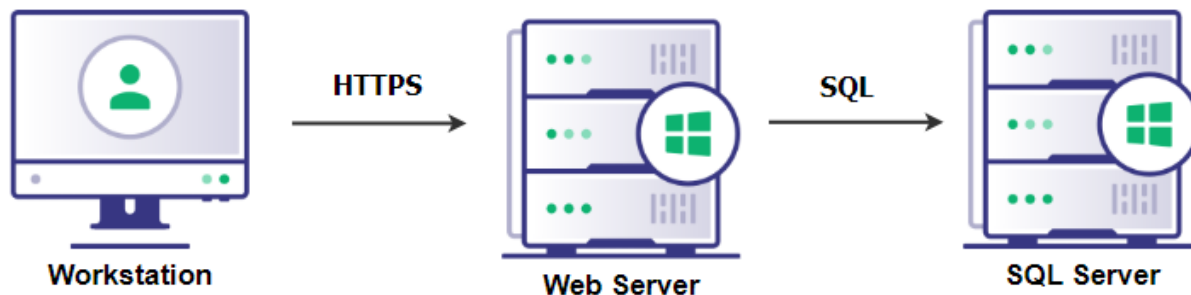


One caveat to this attack, is PAC validation is a feature where the ticket will be sent to the KDC for verification, which could cause this attack to fail.

Delegation Attacks

Kerberos utilizes something called 'delegation', which is when an account can essentially re-use, or "forward", a ticket to another host or application.

For example, in figure X, a user is logged into a web application which uses a SQL DB on another server. Instead of the web application's service account having full access to the entire server the SQL DB is running on, delegation can be configured so that the service account on the web application server can only access the SQL service on the SQL server. In addition, the service account will be used for delegation, meaning it will access the SQL server on the user's behalf and with that user's ticket. This limits both the service account from having complete access to the SQL server, as well as ensuring only authorized users can access the SQL DB through the web application.

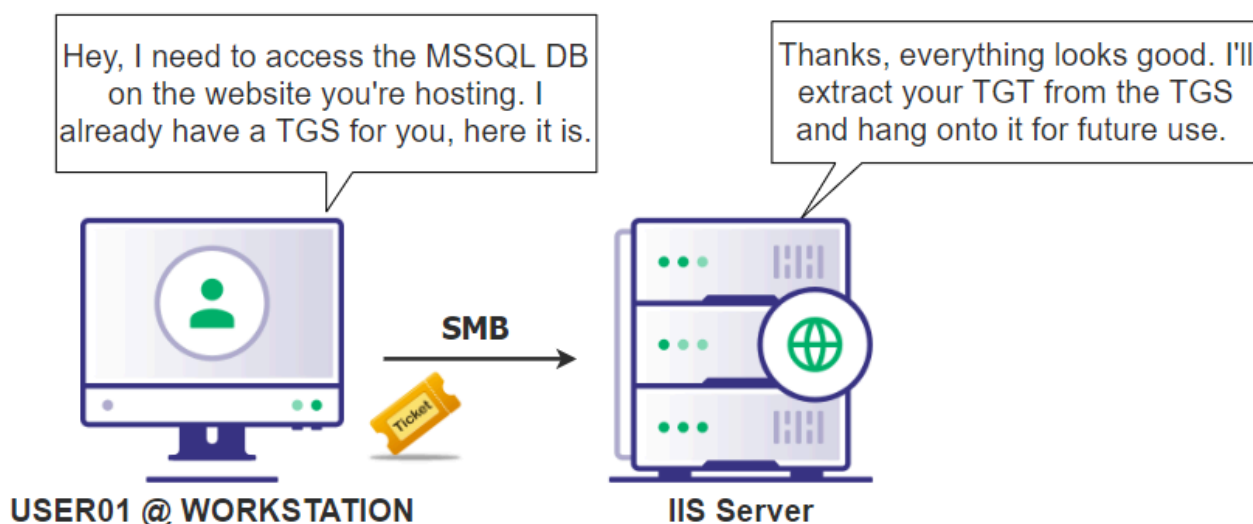
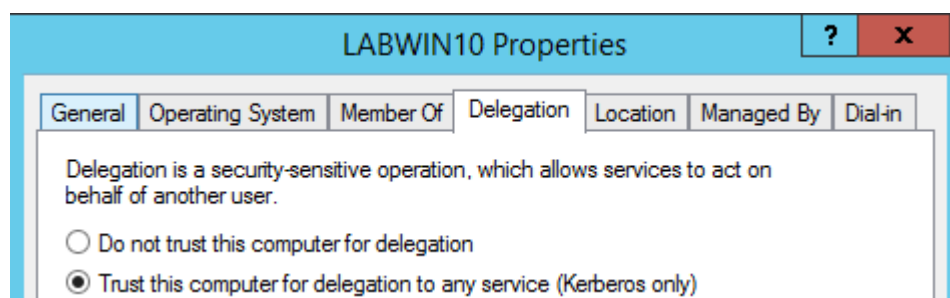


There's three main types of delegation, each with their own attack primitives:

- Unconstrained
- Constrained
- Resource-Based Constrained (RBCD)

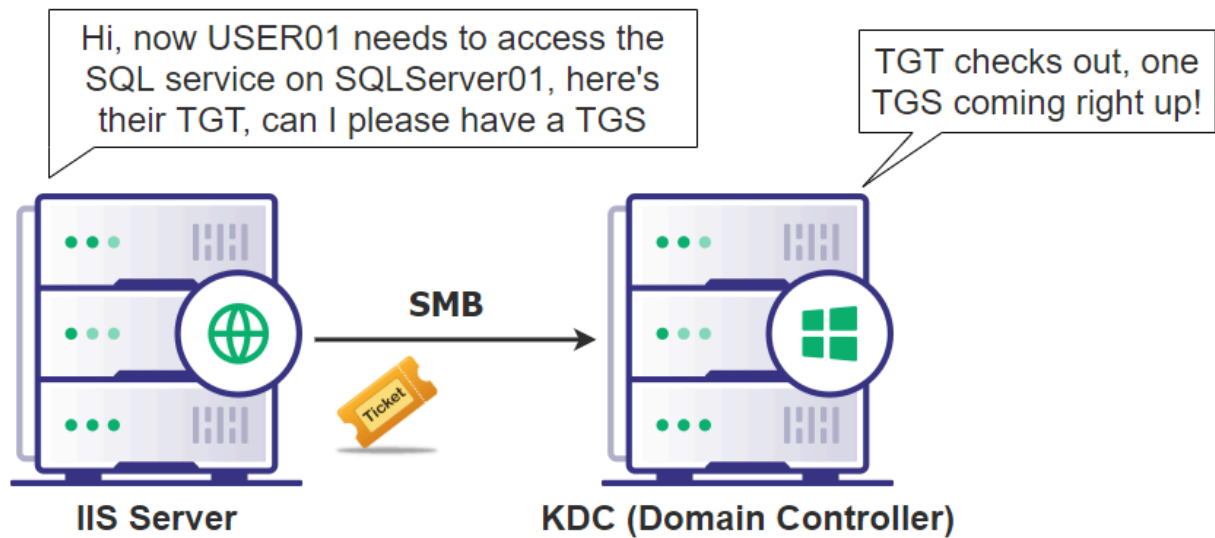
Unconstrained Delegation

Unconstrained Delegation is a very historic way of performing delegation, during Windows 2000. This is configured on the 'Delegation' tab of a computer object within AD.



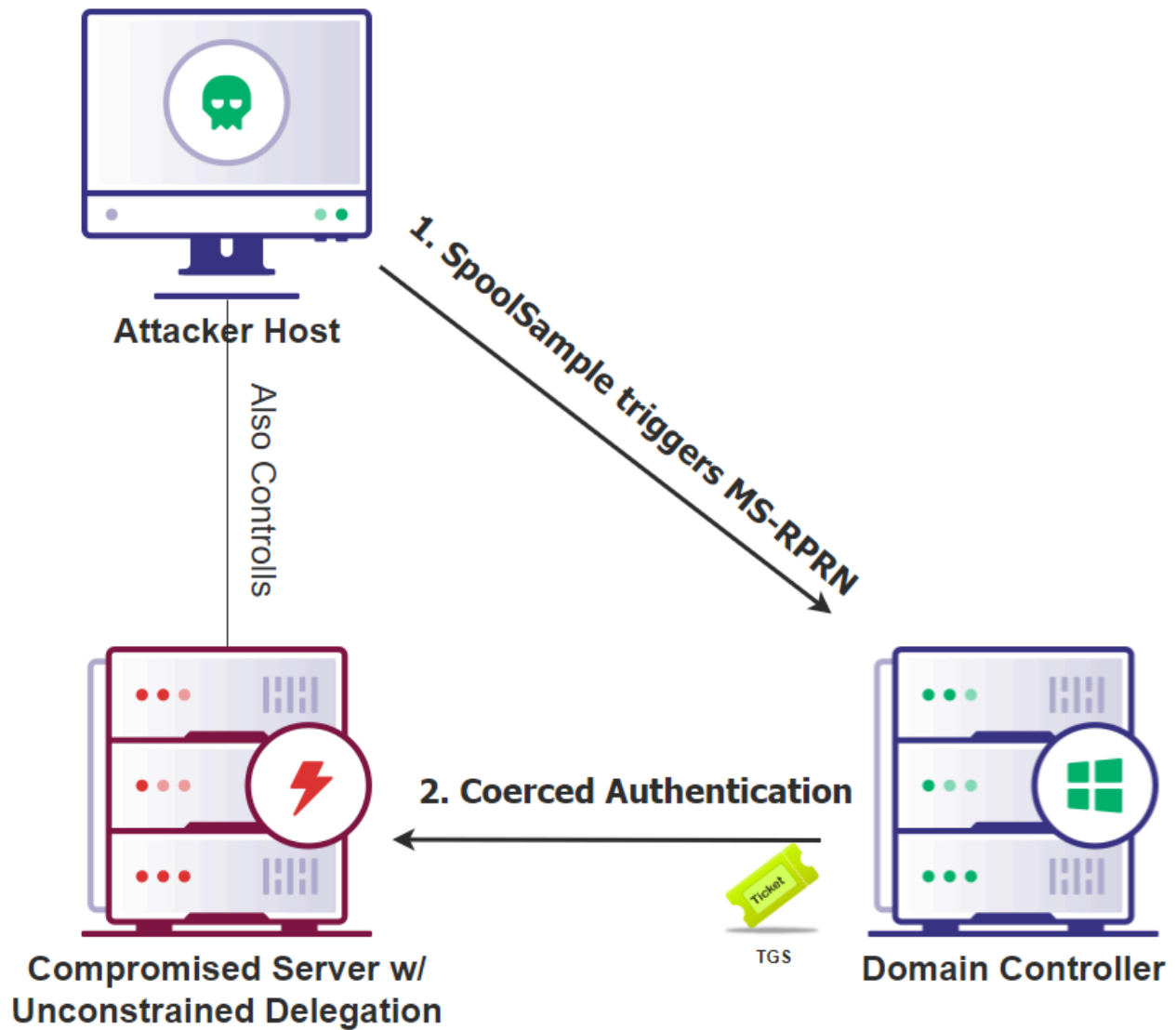
When a machine is configured for unconstrained delegation, any TGS that is sent to the host and contains an SPN, will be accompanied with a TGT and that TGT will be kept in memory for impersonation. The security implication with this, is that if an attacker is

monitoring the memory for Kerberos ticket activity, once a TGS is sent to the host, the attacker can extract the TGT and re-use it.



This can be taken a step further by coercing authentication from any machine in the domain to the unconstrained delegation host via the *printer bug*. The printer bug is a “feature” within the Windows Print System Remote Protocol that allows a host to query another host, asking for an update on a print job. The target host then responds by authenticating to the host that initiated the request, via TGS (which contains a TGT in the case of unconstrained delegation).

What this means, is that if an attacker controls a machine with unconstrained delegation, they could use the printer bug to coerce a domain controller to authenticate to their controlled machine and extract the domain controller’s computer account TGT.

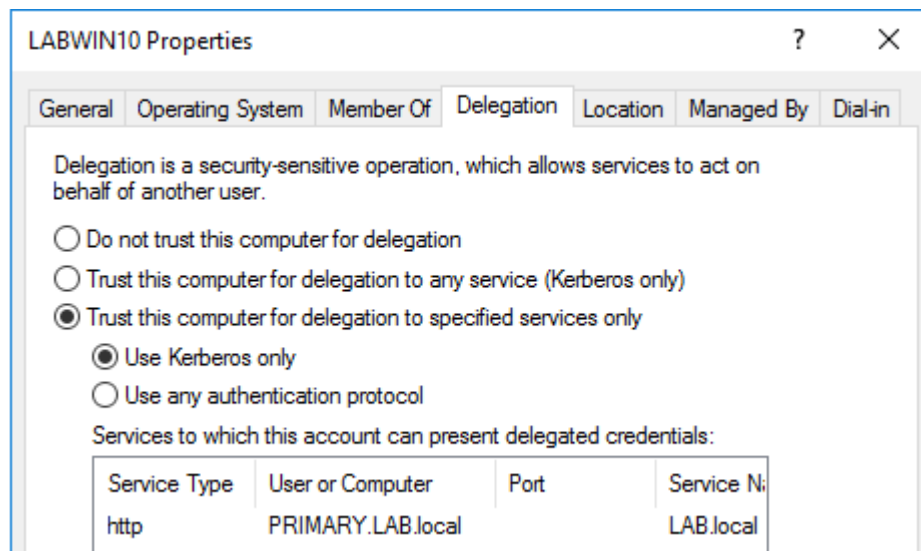


This is possible using Rubeus and SpoolSample.

An important final note is that domain controllers will always have unconstrained delegation enabled by default.

Constrained Delegation

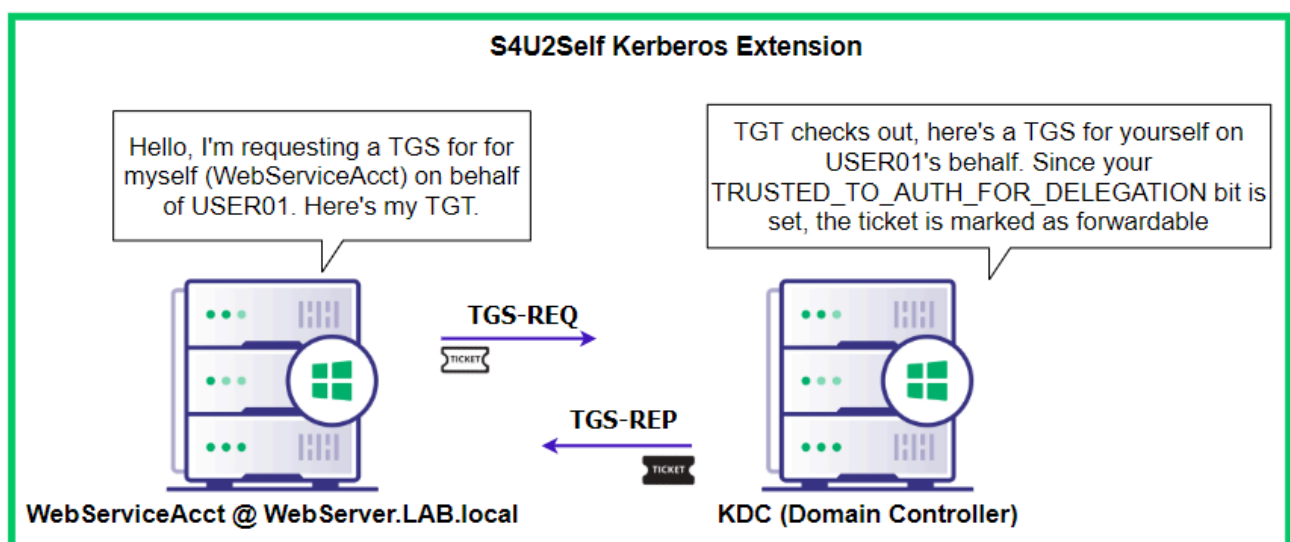
Constrained delegation was introduced during Windows 2003 as an improvement to unconstrained delegation. The major change was that services are limited for an account/machine when impersonating a user (i.e. being delegated). Constrained delegation settings are located in the 'delegation' tab of an object within Active Directory Users and Computers



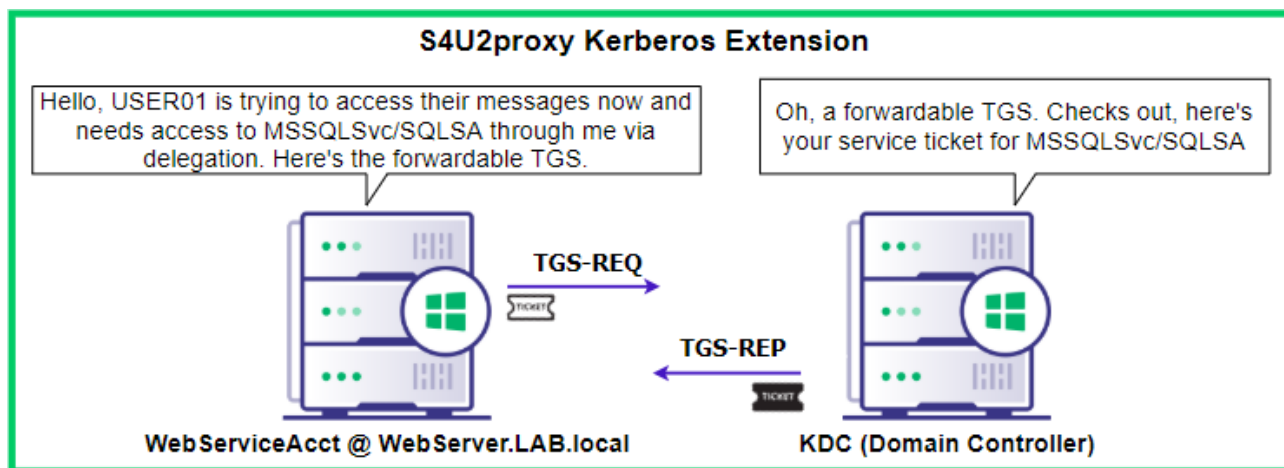
This can also be checked across the domain by looking for the msDS-AllowedToDelegateTo property in accounts/machines via the PowerView function:

```
Get-DomainUser USERNAME -Properties msds-allowedtodelegateto,useraccountcontrol
```

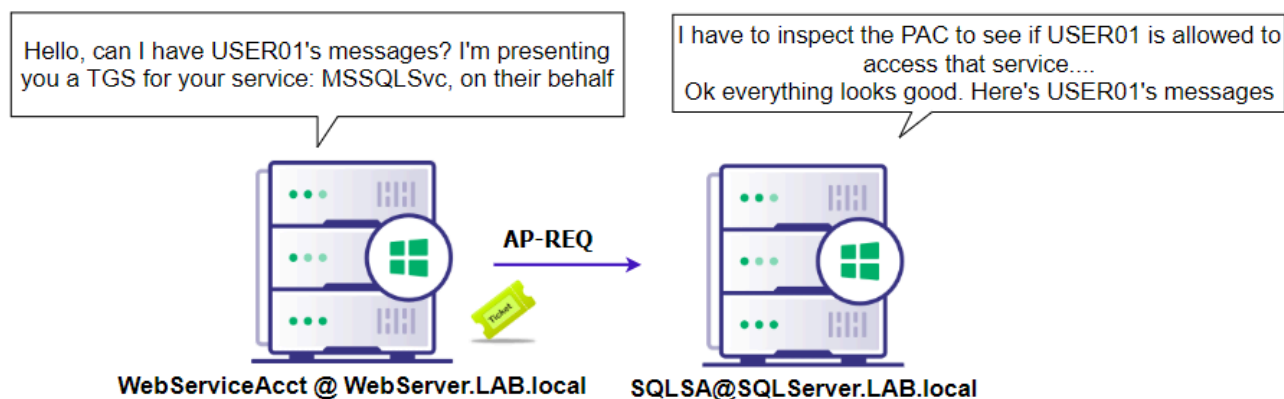
Before using the attack, it's essential to understand how constrained delegation legitimately works. Constrained delegation uses two main Kerberos extensions: S4U2Self and S4U2Proxy. [@harmj0y](#) covered the technical details [here](#), but at a high level, S4U2Self allows an account to request a service ticket to *itself* on behalf of *any* other user (without needing their password). If the TRUSTED_TO_AUTH_FOR_DELEGATION bit is set, The TGS will then be marked as forwardable.



Then the S4U2Proxy is leveraged by the delegated account by using the forwardable TGS to request a TGS to the specified SPN. This is accomplished by the [MS-SFU](#) Kerberos extension which allows a TGS to be requested with a TGS.



Now that service 1 (HTTP/WebServiceAcct) has a ticket for service 2 (MSSQLSvc/SQLSA), service 1 presents that ticket to service 2, who then verifies if the user is allowed to access the service via SIDs within the PAC of the TGS.

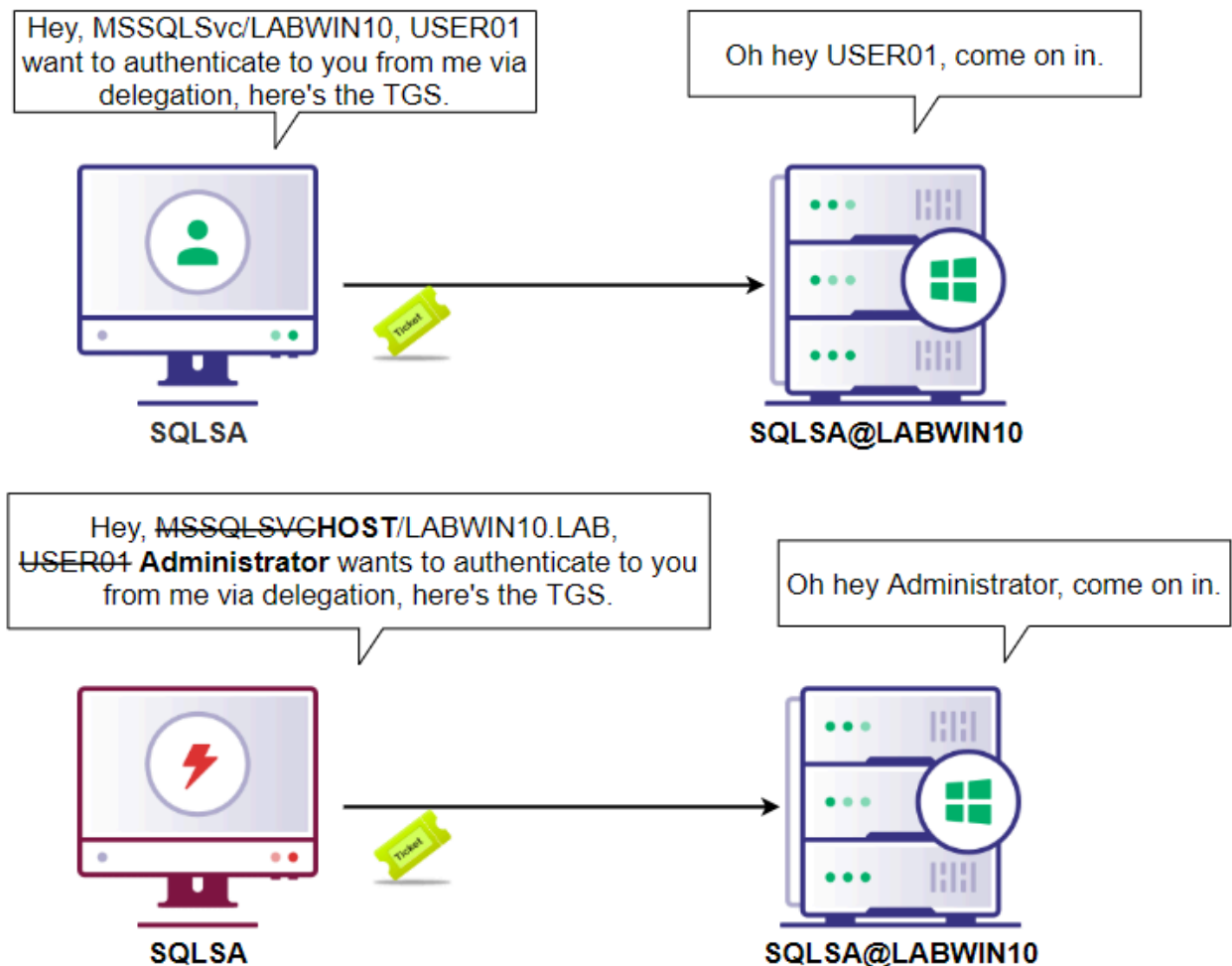


The attack primitive abuses the S4U2Self and S4U2Proxy extensions. If there is an SPN set in the msDS-AllowedToDelegateTo property for an account **and** the userAccountControl property contains the value for 'TRUSTED_TO_AUTH_FOR_DELEGATION', that account can impersonate **any** user to **any** service in that SPN. While it was explained that the S4U2Self extension allows a service to request a TGS to itself on behalf of any user, the additional part of the attack is that the sname (service name) field of the SPN in the (second) TGS is not protected, which allows an attacker to change that to be any service they desire.

The full attack path then looks as followed:

- Attacker Kerberoasts an account (WebSA) that has the msds-AllowedToDelegateTo property set with the SPN of MSSQLSvc/LABWIN10.LAB.local in the property, meaning WebSA can delegate other accounts to access the MSSQLSvc on LABWIN10.LAB.local.
- Rubeus is used to automatically use the S4U2Self extension to request a TGS for the current user, WebSA, on behalf of the user 'Admin'. The returned TGS is marked "forwardable".

- Rubeus then automatically uses the S4U2Proxy extension to use the MS-SFU extension and request a TGS for the delegated SPN, but changing the service portion to whatever the user specifies, e.g. instead of MSSQLSvc/LABWIN10.LAB.local, it requests a TGS for HOST/LABWIN10.LAB.local. Since the service part is not verified, the TGS is returned as the user 'Admin' and SPN HOST/LABWIN10.LAB.local
- The ticket is imported into memory and the user can now access HOST/LABWIN10 as the 'Admin' user, as specified in the TGS.



```
PS C:\> .\Rubeus.exe s4u /user:WebSA /domain:LAB.local /rc4:fbdc5041c96ddbd82224270b57f11fc /impersonateuser:Administrator /msdsspn:"time/PRIMARY.LAB.local" /altservice:cifs /ptt
```

Which user will be impersonated

s4u function

User that is configured for constrained delegation

Original SPN in the ms-allowedtodelegateto field

New service/Service you want to access

One final note on constrained delegation:

TRUSTED_TO_AUTH_FOR_DELEGATION is needed for S4U2Self, but not present by default when adding an SPN for delegation on an account. It can be modified/added if you have the SeEnableDelegationPrivilege over a domain controller.

Resource-Based Constrained Delegation

Resource-Based Constrained Delegation (RBCD) is an improvement on constrained delegation and introduced with Windows Server 2012. The major change in delegation, is that instead of specifying an SPN in the 'Delegation' tab of an account, the delegation settings are now controlled by the *resource* instead. In the previous example with constrained delegation, this would mean that delegation is configured on the backend SQL service, instead of the web service account that delegates to the SQL service.

Where constrained delegation sets the SPN in the msDS-AllowedToDelegateto property, RBCD uses the msDS-AllowedToActOnBehalfOfOtherIdentity property on a computer object. [Elad Shamir](#) did an excellent [write-up](#) on how this can be abused. The summary of the article, is that if the TRUSTED_TO_AUTH_FOR_DELEGATION userAccountControl flag is not present, S4U2Self will still work, but the returned service ticket will not be marked forwardable. In the context of traditional constrained delegation, this means it couldn't be used in the S4U2Proxy extension. However, with RBCD, even if the ticket is not marked as forwardable, it still works.

The attack primitive then, is if an attacker has control of an account with an SPN set and there's a computer account that has the msDS-AllowedToActOnBehalfOfOtherIdentity property set, the computer can be compromised. In addition, if the attacker has GenericWrite privileges over the computer account, they can compromise the computer due to being able to modify the 'AllowedToAct' attribute and setting it to an SPN they control.

If an attacker does not have an account with an SPN set, they can create one by creating a computer object. By default, standard users in AD can create up to 10 computer objects. This can be done with Kevin Robertson's [PowerMad](#) project.

The attack path then looks like this:

- Attacker discovers they have GenericWrite privileges over a computer
- If the attacker doesn't have an account with an SPN set, PowerMad is used to create a machine account, so now the attacker has an account with an SPN.

OR

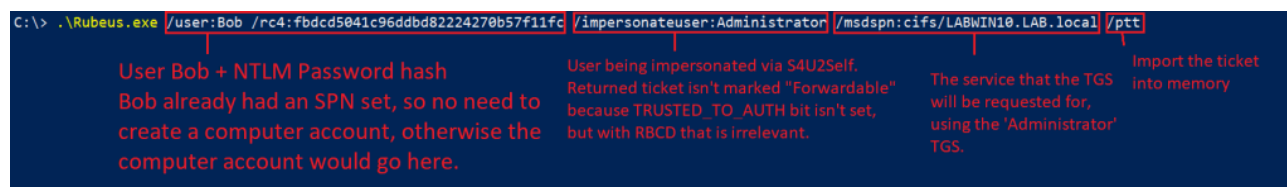
Attacker discovers the msDS-AllowedToActOnBehalfOfOtherIdentity on a computer is set for an SPN that the user has already compromised

THEN

- Rubeus' S4U function is used to request a ticket on behalf of any user, via S4U2Self, to the account with an SPN set (e.g. Getting a TGS for Administrator for newmachine\$)

- Rubeus' S4U function then uses S4U2Proxy to request a TGS as Administrator (with the TGS from S4U2Self) to the target machine. The ticket is not marked as forwardable, which under traditional constrained delegation, would fail, but under RBCD, it does not matter and succeeds.
- The attacker now is able to access the target computer

In figure X, the attacker has compromised Bob, a user account with an SPN set.



For a transcript of the commands used, reference @harmj0y's gist [here](#) (I slightly modified some commands for the User SPN instead of computer account).

Resources:

- <http://www.harmj0y.net/blog/activedirectory/the-most-dangerous-user-right-you-probably-have-never-heard-of/>
- <https://www.harmj0y.net/blog/redteaming/another-word-on-delegation/>
- <https://blogs.uw.edu/kool/2016/10/26/kerberos-delegation-in-active-directory/>